Computer Graphics

CH3 – Basic Transformation and Clipping

Outline

- **Transformations and Clipping**
  - **Basic Transformations**
    - **Translation, Scaling, Rotation**
  - **Composite Transformation**
  - **Clipping Algorithms**

Introduction

- Using output primitives and their attributes, we can create variety of pictures and graphs - also a need for altering or manipulating displays.
- Animations are produced by moving the "camera" or the objects in a scene along animation paths.
- Changes in ***orientation***, ***size***, and ***shape*** are accomplished with ***geometric transformations*** that alter the coordinate descriptions of objects.
- The basic geometric transformations are ***translation***, ***rotation***, and ***scaling***.
- Other transformations that are often applied to objects include ***reflection*** and ***shear***.

Basic Transformations

- Here, we first discuss general procedures for applying ***Translation***, ***Rotation***, and ***Scaling*** parameters to ***reposition*** and ***resize*** two-dimensional objects.
- Then, we consider how transformation equations can be expressed in a more convenient matrix formulation that allows efficient combination of object transformations.

Basic Transformations
Translation

- A **translation** is applied to an object by repositioning it along a straight-line path from one coordinate location to another.
  - The point (x, y) is translated to the point (x', y') by adding the translation distances (t_x, t_y):

$$x' = x + t_x$$
$$y' = y + t_y$$

  - The above equations can be expressed in matrix form by:

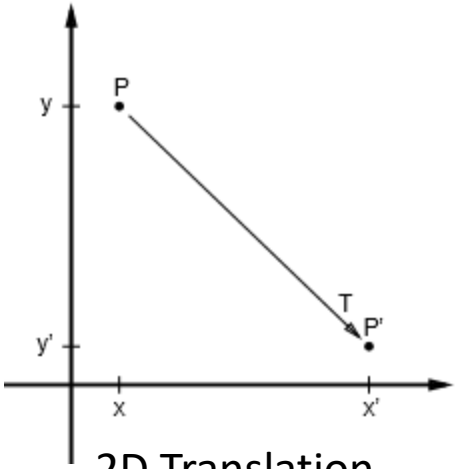
$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

  - So, the translation of the two dimensional vector P by T into P' is given by:

$$P' = P + T$$

where

$$P' = \begin{bmatrix} x' \\ y' \end{bmatrix}, \quad P = \begin{bmatrix} x \\ y \end{bmatrix}, \quad \text{and} \quad T = \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$



2D Translation

Basic Transformations
Scaling relative to origin

- A **scaling** transformation alters the size of an object by scaling factors $\boldsymbol{s_x}$ and $\boldsymbol{s_y}$.
  - The vertex (x, y) is scaled into the vertex (x', y') by multiplying it with the scaling factors $\boldsymbol{s_x}$ and $\boldsymbol{s}_y$:

$$x' = s_x x$$
$$y' = s_y y$$

  - This can be expressed in matrix form by:

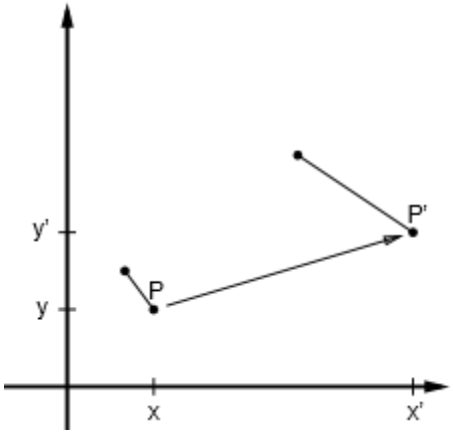
$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

or

$$P' = S \cdot P$$

where

$$S = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$$



2D Scaling

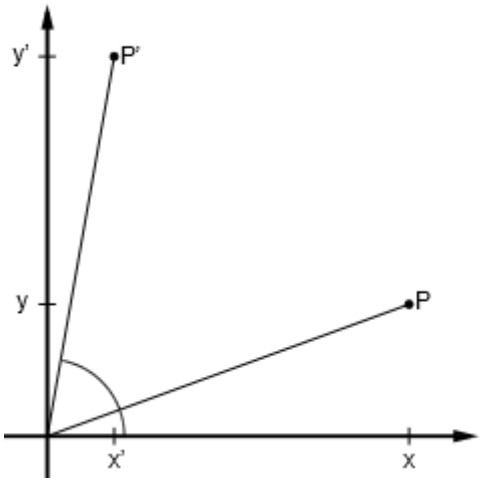
- If $s_x = s_y$ the transformation is called a ***uniform scaling.***
- If $s_x \neq s_y$ it is called a ***differential scaling***.

Basic Transformations
Rotation relative to origin

- The point (x, y), or (r, φ) in polar coordinates, is rotated anti-clockwise about the origin by θ into the point (x', y'), or $(r, \varphi + \theta)$.
  - So
$$x' = rcos(\varphi + \theta) = r(\cos\varphi\cos\theta - \sin\varphi\sin\theta)$$
$$y' = rsin(\varphi + \theta) = r(\cos\varphi\sin\theta + \sin\varphi\cos\theta)$$
  - Now,
$$x = rcos\varphi \quad \text{and} \quad y = rsin\varphi$$
  - Therefore
$$x' = xcos\theta - y\sin\theta$$
$$y' = xsin\theta + y\cos\theta$$
  - This can be expressed by:
$$\boldsymbol{P'} = \boldsymbol{R} \cdot \boldsymbol{P}$$

where

$$R = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$



2D Rotation

Basic Transformations

- All of the three basic transformations are ***rigid body transformations*** – that moves objects without deformation.
- That is, every point on the object is translated, scaled or rotated by the same amount.
  - A ***straight Line segment*** is *translated*, *scaled* or *rotated* by applying the transformation to each of the line endpoints and redrawing the line between the new endpoint positions.
  - ***Polygons*** are *translated*, *scaled* or *rotated* by adding the transformation vectors to the coordinate position of each vertex and regenerating the polygon using the new set of vertex coordinates and the current attribute settings.

Homogeneous Coordinates

- Many graphics applications involve sequences of geometric transformations.
  - E.g., An animation might require an object to be translated and rotated at each increment of the motion.
- In design and picture construction applications, we perform translations, rotations, and scaling to fit the picture components into their proper positions.
- We consider how the matrix representations can be reformulated so that such transformation sequences can be efficiently processed.

Homogeneous Coordinates

- A combination of *translations*, *rotations* and *scaling* can be expressed as:

$$\begin{aligned} P' &= S \cdot R \cdot (P + T) \\ &= (S \cdot R) \cdot P + (S \cdot R \cdot T) \\ &= M \cdot P + A \end{aligned}$$

- Because scaling and rotation are expressed as matrix multiplication but translation is expressed as matrix addition,
  - it is not, in general, possible to combine a set of operations into a single matrix operation.
- ***Composition of transformations*** is often desirable if the same set of operations have to be applied to a list of position vectors.

Homogeneous Coordinates

- We can solve this problem by representing points in ***homogenous coordinates***.
- In homogenous coordinates we add a third coordinate to a point, i.e., instead of representing a point by a pair (x, y), we represent it as (x, y, W).
- Two sets of homogenous coordinates represent the same point if one is a multiple of the other.
  - E.g. $(2, 3, 6)$ and $(4, 6, 12)$ represent the same point.

- Given homogenous coordinates (x, y, W), the Cartesian coordinates (x', y') correspond to $(\frac{x}{W}, \frac{y}{W})$, i.e. the homogenous coordinates at which W = 1.
  - Points with $W = 0$ are called points at infinity.

Homogeneous Coordinates

- Representing points as 3-dimensional vectors, we can re-write our transformation matrices as follows:
- **Translation**
  - A translation by (t_x, t_y) is represented by:

$$\boldsymbol{P}' = \boldsymbol{T}(\boldsymbol{t_x}, \boldsymbol{t_y}) \cdot \boldsymbol{P}$$

    where

$$\boldsymbol{T}(\boldsymbol{t_x}, \boldsymbol{t_y}) = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

- **Scaling**
  - Scaling by the factors $\boldsymbol{s_x}$ and $\boldsymbol{s_y}$ relative to the origin is given by:

$$\boldsymbol{P}' = \boldsymbol{S}(\boldsymbol{s_x}, \boldsymbol{s_y}) \cdot \boldsymbol{P}$$

    where

$$\boldsymbol{S}(\boldsymbol{sx}, \boldsymbol{s_y}) = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Homogeneous Coordinates

- Representing points as 3-dimensional vectors, we can re-write our transformation matrices as follows:
- **Rotation**
  - Rotation about the origin by θ is given by:

$$\boldsymbol{P}' = \boldsymbol{R}(\boldsymbol{\theta}) \cdot \boldsymbol{P}$$

where

$$\boldsymbol{R}(\boldsymbol{\theta}) = \begin{bmatrix} cos\theta & -sin\theta & 0 \\ sin\theta & cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Composition of 2D Transformations

- Using the associative property of matrix multiplication, we can combine several transformations into a single matrix.
- It can easily be shown that:

$$T\left(t_{x_2}, t_{y_2}\right) \cdot T\left(t_{x_1}, t_{y_1}\right) = T\left(t_{x_1} + t_{x_2}, t_{y_1} + t_{y_2}\right)$$

$$S(s_{x_2}, s_{y_2}) \cdot S(s_{x_1}, s_{y_1}) = S(s_{x_1} s_{x_2}, s_{y_1} s_{y_2})$$

$$R(\theta_2) \cdot R(\theta_1) = R(\theta_1 + \theta_2)$$

Composition of 2D Transformations

General fixed-point Scaling

- Scaling with factors s_x and s_y with respect to the fixed point (x, y) can be achieved by the following list of operations:
  1. Translate by $(-x, -y)$ so that the fixed point is moved to the origin;
  2. Scale by s_x and s_y with respect to the origin;
  3. Translate by (x, y) to return the fixed point back to its original position.
- So, the general fixed-point scaling transformation matrix is:

$$S_{x,y}\,(s_x, s_y) = T(x, y) \cdot S(s_x, s_y) \cdot T(-x, -y)$$

$$= \begin{bmatrix} s_x & 0 & x(1 - s_x) \\ 0 & s_y & y(1 - s_y) \\ 0 & 0 & 1 \end{bmatrix}$$

Composition of 2D Transformations

General pivot-point rotation

- A rotation about the rotation-point or pivot-point (x, y) by θ is achieved as follows:
  1. Translate by $(-x, -y)$;
  2. Rotate by θ;
  3. Translate by (x, y).
- The general pivot-point rotation matrix is therefore given by:

$$R_{x,y}(\theta) = T(x, y) \cdot R(\theta) \cdot T(-x, -y)$$

$$= \begin{bmatrix} cos\theta & -sin\theta & x(1 - cos\theta) + ysin\theta \\ sin\theta & cos\theta & y(1 - cos\theta) - xsin\theta \\ 0 & 0 & 1 \end{bmatrix}$$

Composition of 2D Transformations

Inverse transformations

- The matrix of an inverse transformation is the inverse matrix of the transformation:
  - If $P' = M \cdot P$ then $P = M^{-1} \cdot P'$
  - It can easily be shown that:

$$T(t_x, t_y)^{-1} = T(-t_x, -t_y)$$

$$S(s_x, s_y)^{-1} = S\left(\frac{1}{s_x}, \frac{1}{s_y}\right)$$

$$R(\theta)^{-1} = R(-\theta)$$

Clipping Algorithms

- Usually only a specified region of a picture needs to be displayed.
  - This region is called the **clip window**.
- An algorithm which displays only those primitives (or part of the primitives) which lie inside a clip window is referred to as a ***clipping algorithm***.

Clipping Points

- A point (x, y) lies inside the rectangular clip window $(x_{min}, y_{min}, x_{max}, y_{max})$ if and only if the following inequalities hold:

$$x_{min} \leq x \leq x_{max}$$
$$y_{min} \leq y \leq y_{max}$$

Clipping Algorithms
Line Clipping

The Cohen-Sutherland algorithm for line clipping

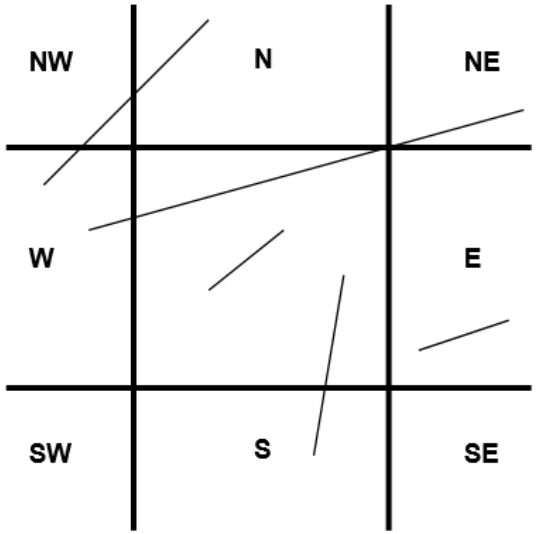
- Clipping a straight line against a rectangular clip window results in either:
  1. A line segment whose endpoints may be different from the original ones, or
  2. Not displaying any part of the line. This occurs if the line lies completely outside the clip window.
- The Cohen-Sutherland line clipping algorithm considers each endpoint at a time and truncates the line with the clip window's edges until the line can be trivially accepted or trivially rejected.
- A line is trivially rejected if both endpoints lie on the same outside half-plane of some clipping edge

Clipping Algorithms

The Cohen-Sutherland algorithm for line clipping

- The xy plane is partitioned into nine segments by extending the clip window's edges.
- Each segment is assigned a 4-bit code according to where it lies with respect to the clip window:

Partition of plane into 9 segments.



$\boldsymbol{Bit}$	$\boldsymbol{Side}$	$\boldsymbol{Inequality}$
1	N	$y > y_{max}$
2	S	$y < y_{min}$
3	E	$x > x_{max}$
4	W	$x < x_{min}$

  - For example bit 2 is set if and only if the region lies to the south of (below) the clip window.

Clipping Algorithms

The Cohen-Sutherland algorithm for line clipping

- The Cohen-Sutherland algorithm starts by assigning an ***outcode*** to the two line endpoints (say c_1 and c_2).
  - If both *outcodes* are 0 ($\boldsymbol{c_1\ OR\ c_2 = 0}$) the line lies entirely in the clip window and is thus trivially accepted.
  - If the two *outcodes* have at least one bit in common ($\boldsymbol{c_1\ AND\ c_2 \neq 0}$), then they lie on the same side of the window and the line is trivially rejected.
  - If a line cannot be accepted or rejected, it is then truncated by an intersecting clip edge and the previous steps are repeated.

Clipping Algorithms

The Cohen-Sutherland algorithm for line clipping

<u>Algorithm</u>:

Step 1: Assign a region code for each end points.

Step2: If both endpoints have a region code 0000 then accept the line. Else apply Step 3.

Step3: Perform the logical AND operation for both region codes.

- If the result is not 0000, then reject the line.
- Else clip the line by:
  - Choose an endpoint of the line that is outside the window.
  - Find the intersection point at the window boundary (based on region code).
  - Replace endpoint with the intersection point and update the region code.

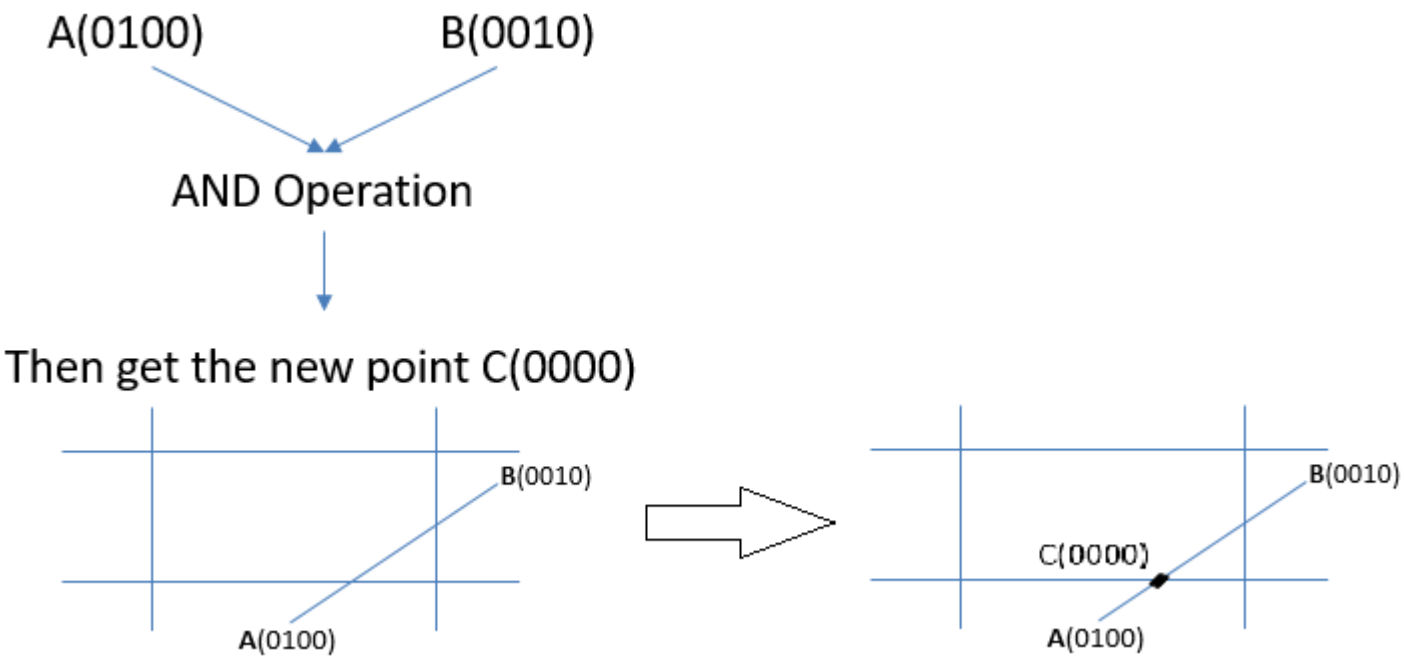
Step 4: Repeat Step 2 until we find a clipped line either trivially accepted or rejected.

Step 5: Repeat Step 1 for other lines.

Clipping Algorithms

The Cohen-Sutherland algorithm for line clipping

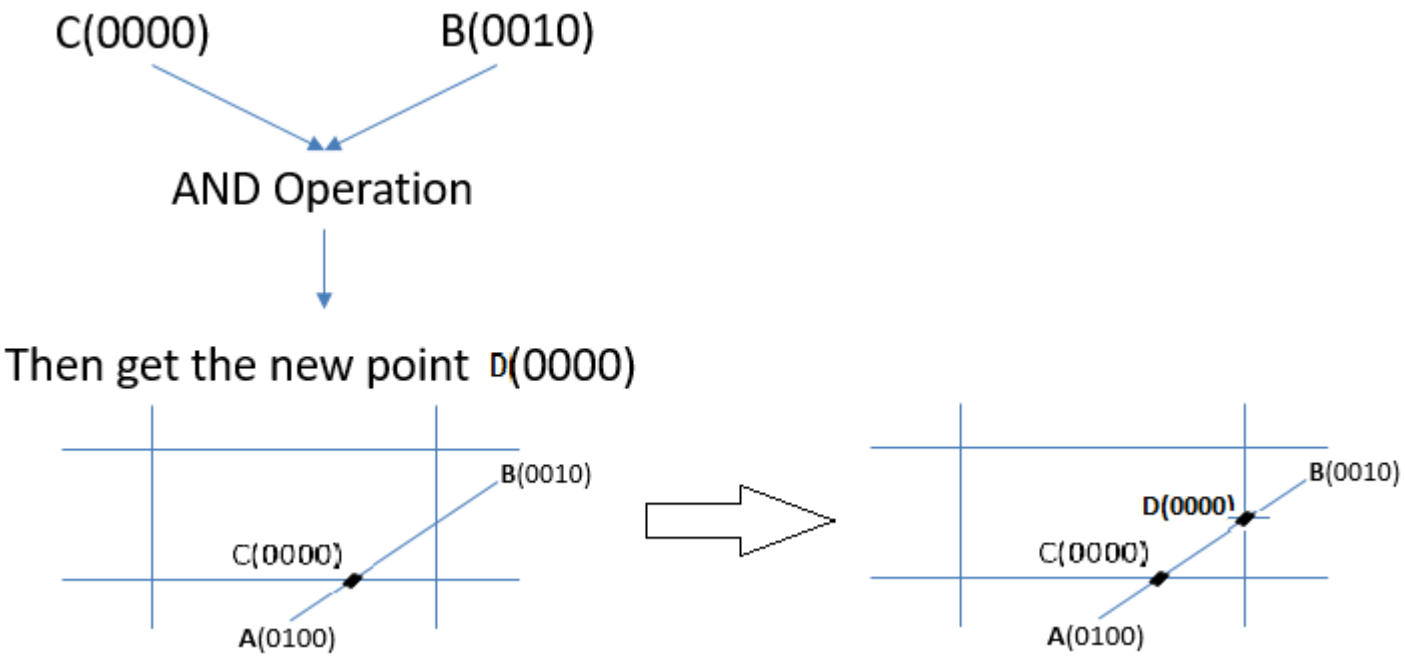
<u>Example</u>:



Clipping Algorithms

The Cohen-Sutherland algorithm for line clipping

<u>Example</u>:



Then we have the final line after clipping CD.

Clipping Algorithms
Polygon Clipping

Sutherland-Hodgman polygon clipping algorithm

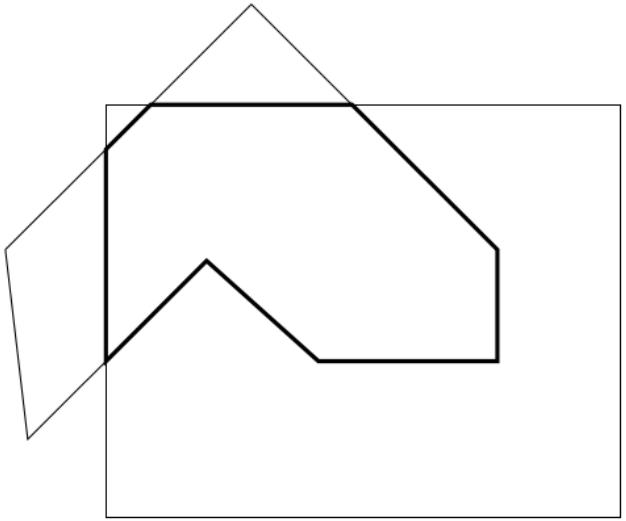
- A polygon can be clipped against a rectangular clip window by considering each clip edge at a time.

$$Left \rightarrow Right \rightarrow Bottom \rightarrow Top$$

Clipping Algorithms
Polygon Clipping

Sutherland-Hodgman polygon clipping algorithm

- For filled polygons, a new polygon boundary must be computed.



- At each clipping stage, a new polygon is created by removing the outside vertices and inserting the intersections with the clip boundary.

Clipping Algorithms
Polygon Clipping

Sutherland-Hodgman polygon clipping algorithm

Clipping against an edge

- Given the polygon $[v_1; v_2; \ldots; v_n]$, we can create a new polygon $[w_1; w_2; \ldots; w_m]$ by
  - considering the vertices in sequence and
  - deciding which ones (and any intersections with the edge) have to be inserted into the clipped polygon vertex list.
- Suppose that v_i has been processed in the previous step.
- There are four possible cases which need to be considered while processing the next vertex v_{i+1}.

Clipping Algorithms
Polygon Clipping

Sutherland-Hodgman polygon clipping algorithm

Clipping against an edge

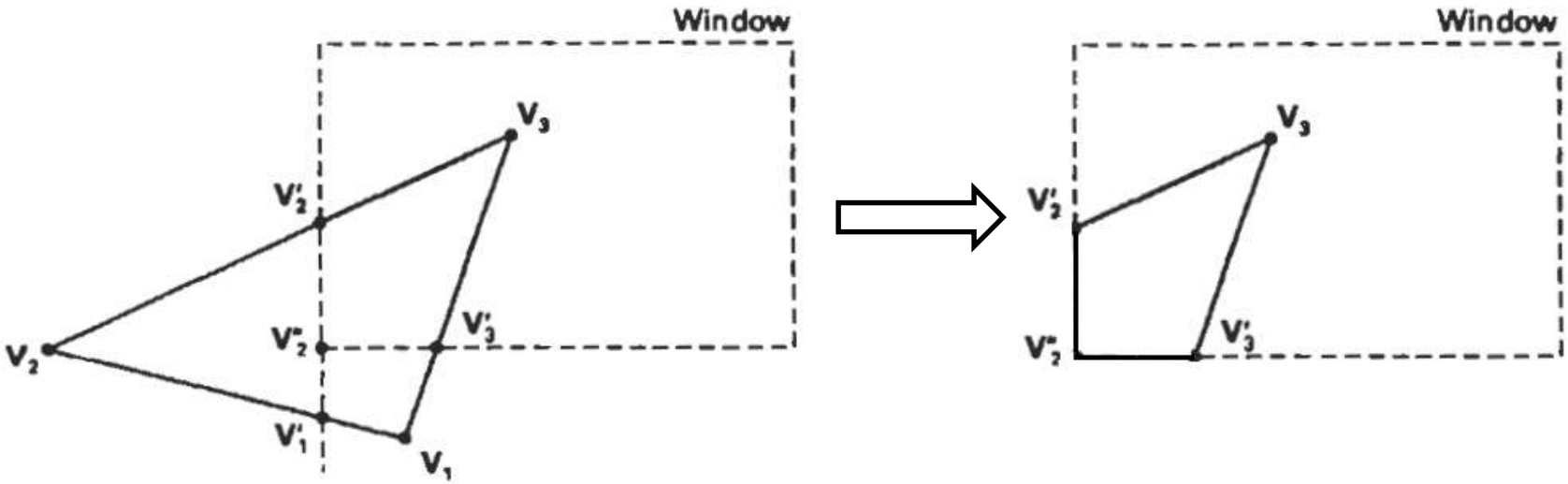
1. If v_i is outside the window and v_{i+1} is inside, then the intersection of the line $v_i \rightarrow v_{i+1}$ with the clipping edge, and the vertex v_{i+1} have to be inserted into the new polygon list.
2. If v_i and v_{i+1} are both outside the clip window then no point need be inserted.
3. If both v_i and v_{i+1} are inside the window, v_{i+1} is inserted into the list, and finally.
4. If v_i is inside the window and v_{i+1} is outside, only the intersection of the line $v_i \rightarrow v_{i+1}$ with the clip boundary is inserted.

Clipping Algorithms
Polygon Clipping

Sutherland-Hodgman polygon clipping algorithm

Clipping against an edge

- **Example**: Processing the vertices of the polygon below through a boundary-clipping pipeline.

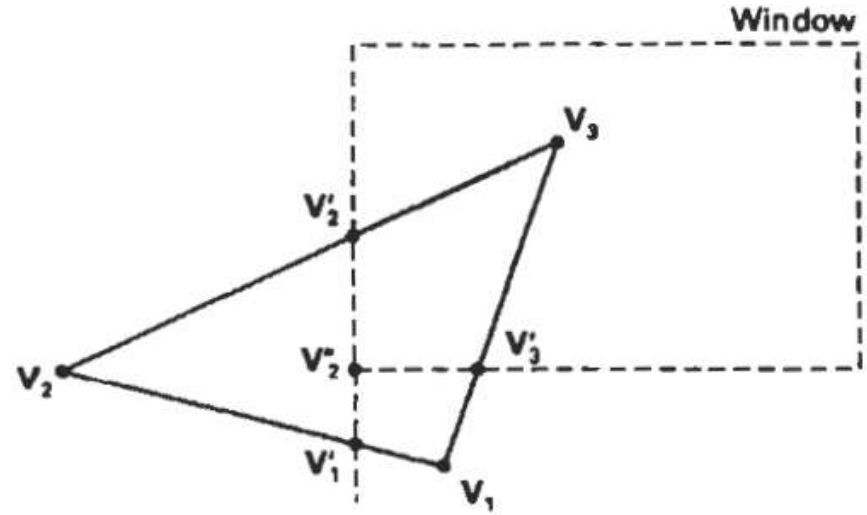


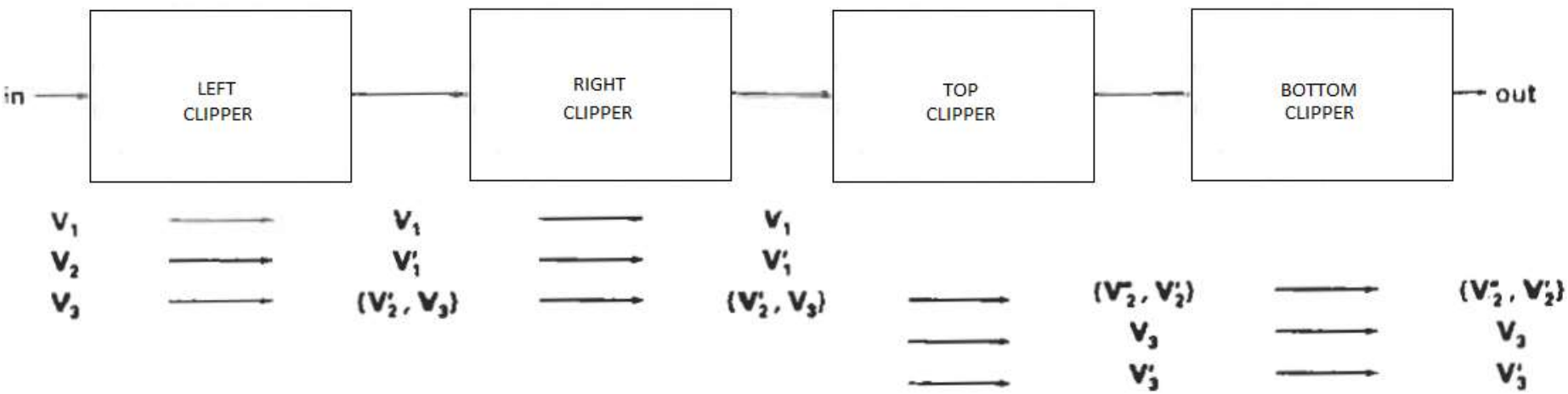
- After all vertices are processed through the pipeline, the vertex list for the clipped polygon is $[v_2'', v_2', v_3, v_3']$.

Clipping Algorithms
Polygon Clipping

Sutherland-Hodgman polygon Clipping against an edge

- **Example**: cont….





Thank You!!!

End of CH3!

Computer Graphics

CH4 – Windows and Viewports

Outline

- **Windows and Viewports**
  - **Co-ordinates Representation**
  - **Viewing Pipeline**
  - **Window to Viewport Transformation**
  - **Zooming and Panning effects with windows and viewports**

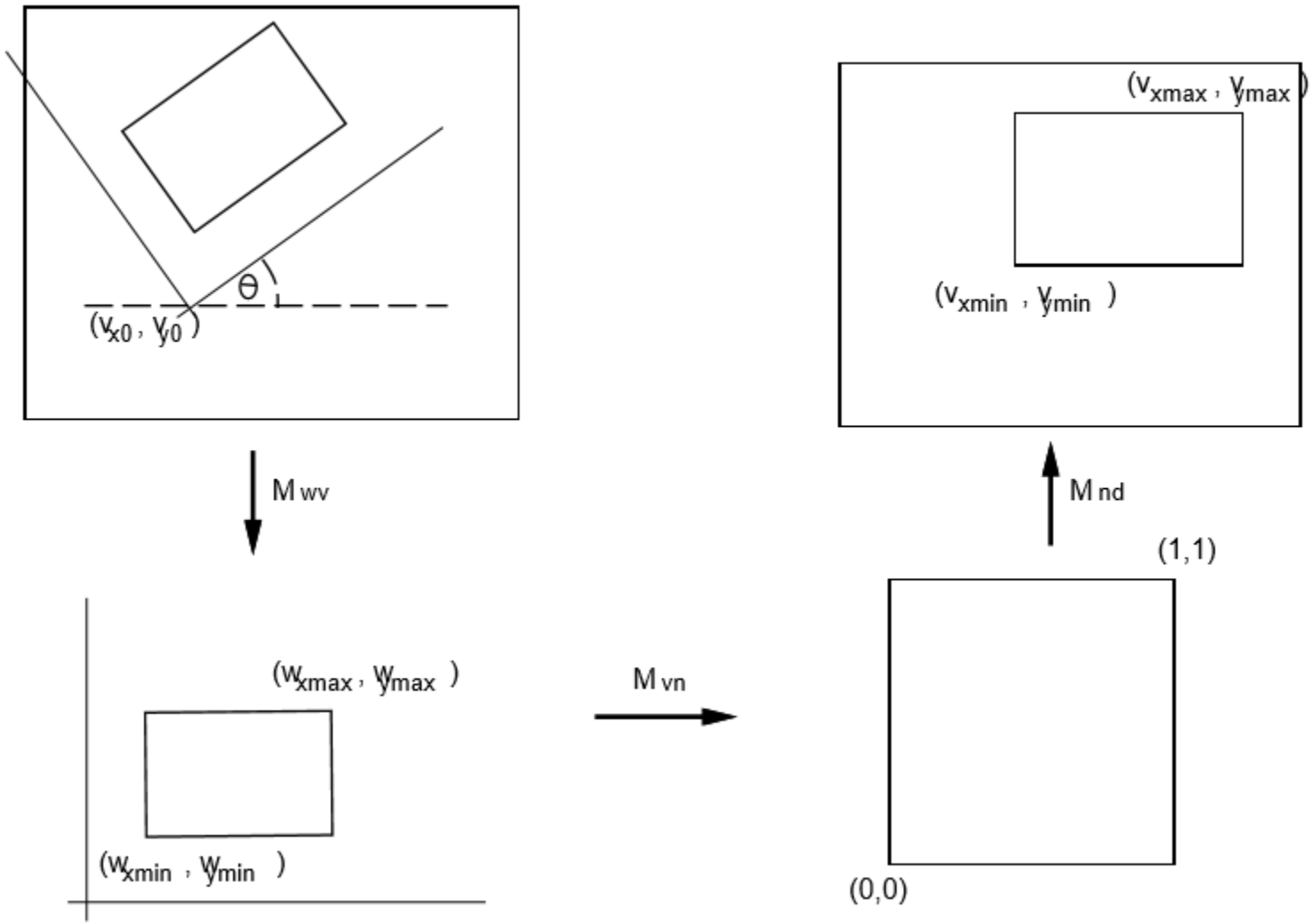
Windows and Viewports Co-ordinates Representation

- Viewing involves transforming an object specified in a world coordinates frame of reference into a device coordinates frame of reference.
  - A region in the world coordinate system which is selected for display is called a **window**.
  - The region of a display device into which a window is mapped is called a ***viewport***.
- The steps required for transforming world coordinates (or modelling coordinates) into device coordinates are referred to as the ***viewing pipeline***.

Windows and Viewports Viewing Pipeline

- Steps involved in the 2D viewing pipeline:
  1. Transforming ***world coordinates*** into ***viewing coordinates***, usually called the ***viewing transformation***.
     - Given 2D objects represented in world coordinates, a window is specified in terms of a viewing coordinate system defined relative to the world coordinate system.
  2. ***Normalizing*** the viewing coordinates.
     - The object's world coordinates are transformed into viewing coordinates and usually normalized so that the extents of the window fit in the rectangle with lower left corner at (0,0) and the upper right corner at (1,1).
  3. Transforming the normalized viewing coordinates into device coordinates.
     - The normalized viewing coordinates are then transformed into device coordinates to fit into the viewport.

Windows and Viewports Viewing Pipeline



Windows and Viewports
Window to Viewport Transformation

- Consider a viewing coordinate system defined as a Cartesian coordinate system with the origin having world coordinates (u_0, v_0).
- The axis of the viewing coordinate system makes an angle of θ with the world coordinate system $x - axis$.
- The transformation M_{wv} transforms a point with world coordinates P_w into viewing coordinates P_v.
- This transformation can be achieved by first translating by $(-u_0, -v_0)$ and then rotating by $-\theta$.
- So

$$P_v = M_{wv} \cdot P_w$$

where

$$M_{wv} = R(-\theta) \cdot T(-u_0, -v_0)$$

Windows and Viewports
Window to Viewport Transformation

- The window is defined as the rectangle having bottom-left corner at (x_l, y_b), the opposite corner at (x_r, y_t), and edges parallel to the viewing coordinate axes.
- The ***normalizing transformation*** M_{vn} is obtained by first translating by $(-x_l, -y_b)$ and then scaling with factors $\frac{1}{x_r - x_l}$ and $\frac{1}{y_t - y_b}$, so that the lower left and upper right vertices of the window have normalized viewing coordinate $(0,0)$ and $(1,1)$ respectively.
- So a point with viewing coordinates P_v has normalized viewing coordinates:

$$P_n = M_{vn} \cdot P_v$$

where

$$M_{vn} = S\left(\frac{1}{x_r - x_l}, \frac{1}{y_t - y_b}\right) \cdot T(-x_l, -y_b)$$

Windows and Viewports
Window to Viewport Transformation

- Transforming into the device coordinates is accomplished by the matrix M_{nd}.
- If the viewport is a rectangle with the lower left and upper right corners having device coordinates (u_l, v_b) and (u_r, v_t) respectively, then M_{nd} is achieved by first scaling with factors $(u_r - u_l)$ and $(v_t - v_b)$ and then translating by (u_l, v_b):

$$P_d = M_{nd} \cdot P_n$$

where

$$M_{nd} = T(u_l, v_b) \cdot S(u_r - u_l, v_t - v_b)$$

Windows and Viewports
Window to Viewport Transformation

- Thus, the whole viewing pipeline can be achieved by concatenating the three matrices:

$$M_{wd} = M_{nd} \cdot M_{vn} \cdot M_{wv}$$

$$= T(u_l, v_t) \cdot S(u_r - u_l, v_t - v_b) \cdot S\left(\frac{1}{x_r - x_l}, \frac{1}{y_t - y_b}\right) \cdot R(-\theta) \cdot T(-u_0, -v_0)$$

$$= T(u_l, v_t) \cdot S\left(\frac{u_r - u_l}{x_r - x_l}, \frac{v_t - v_b}{y_t - y_b}\right) \cdot R(-\theta) \cdot T(-u_0, -v_0)$$

- Defining

$$S_x = \frac{u_r - u_l}{x_r - x_l}$$

$$S_y = \frac{v_t - v_b}{y_t - y_b}$$

Windows and Viewports
Window to Viewport Transformation

- If $s_x \neq s_y$, M_{wd} scales objects differentially;
- However if we want objects to be scaled uniformly during the viewing process and that all objects in the window are mapped into the viewport, then we define

$$s' = \min(s_x, s_y)$$

and scale using $S(s', s')$ instead of $S(s_x, s_y)$.